Pour implémenter une authentification **OAuth2 avec JWT (JSON Web Tokens)** propre et compatible avec le bouton **"Authorize"** de Swagger UI, nous allons procéder en **5 étapes**.

Étape 1 : Installer les dépendances JWT

Dans ton terminal, installe `pyjwt` (le standard moderne et maintenu pour les JWT en Python) ainsi que `python-multipart` (obligatoire pour que FastAPI puisse lire les formulaires `OAuth2PasswordRequestForm`) :

```
uv add pyjwt python-multipart
```

Étape 2 : Ajouter la configuration JWT dans `.env` et `config.py`

1. Ajoute la clé secrète, l'algorithme et la durée de validité dans ton fichier **`.env`** à la racine du projet :

```
SECRET_KEY="ta_cle_secrete_super_longue_et_aleatoire_a_changer_en_prod"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

(Astuce : tu peux générer une vraie clé sécurisée dans ton terminal avec `openssl rand -hex 32`).

2. Mets à jour **`src/project_manager/core/config.py`** pour charger ces variables :

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    PROJECT_NAME: str = "Project Manager API"
    MONGO_URI: str
    MONGO_DB_NAME: str = "taskflow_db"

    # Configuration JWT
    SECRET_KEY: str
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")


settings = Settings()
```

Étape 3 : Créer la génération de Token dans `core/security.py`

Complète ton fichier **`src/project_manager/core/security.py`** avec la fonction `create_access_token` :

```
from datetime import datetime, timedelta, timezone
from typing import Any, Dict
import bcrypt
import jwt
from project_manager.core.config import settings


def hash_password(password: str) -> str:
    pwd_bytes = password.encode("utf-8")[:72]
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(pwd_bytes, salt).decode("utf-8")


def verify_password(plain_password: str, hashed_password: str) -> bool:
    pwd_bytes = plain_password.encode("utf-8")[:72]
    hashed_bytes = hashed_password.encode("utf-8")
    return bcrypt.checkpw(pwd_bytes, hashed_bytes)


def create_access_token(data: Dict[str, Any], expires_delta: timedelta |
None = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (
        expires_delta or
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

Étape 4 : Créer le schéma du Token (`schemas/auth.py` ou `dto/auth.py`)

Crée le fichier **`src/project_manager/dto/auth.py`** pour typer la réponse de l'endpoint de connexion :

```
from pydantic import BaseModel


class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
```

```
Token.model_rebuild()
```

Étape 5 : Créer la dépendance `get_current_user` et le Router `auth.py`

5.1 Créer `src/project_manager/core/deps.py`

Ce fichier centralise tes dépendances injectables (`DbDep` et `CurrentUserDep`) pour éviter les imports circulaires entre tes routers :

```
from typing import Annotated
import jwt
from bson import ObjectId, errors
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from motor.motor_asyncio import AsyncIOMotorDatabase

from project_manager.core.config import settings
from project_manager.db.mongodb import get_database

# Indique à Swagger UI quelle route génère le token
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")

DbDep = Annotated[AsyncIOMotorDatabase, Depends(get_database)]
TokenDep = Annotated[str, Depends(oauth2_scheme)]


async def get_current_user(token: TokenDep, db: DbDep) -> dict:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Impossible de valider les identifiants (token invalide ou
expiré).",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM]
        )
        user_id: str | None = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except jwt.PyJWTError:
        raise credentials_exception

    try:
        obj_id = ObjectId(user_id)
    except errors.InvalidId:
        raise credentials_exception

    user = await db["users"].find_one({"_id": obj_id})
```

```
    if user is None or not user.get("is_active", True):
        raise credentials_exception

    user["_id"] = str(user["_id"])

    # Peupler le rôle de l'utilisateur connecté
    if "role_id" in user:
        try:
            role = await db["roles"].find_one({"_id":
ObjectId(user["role_id"])})
            if role:
                role["_id"] = str(role["_id"])
                user["role"] = role
        except errors.InvalidId:
            user["role"] = None

    return user


CurrentUserDep = Annotated[dict, Depends(get_current_user)]
```

5.2 Créer `src/project_manager/routers/auth.py`

Ce router gère la connexion (`/login`) et la récupération du profil connecté (`/me`) :

```
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm

from project_manager.core.deps import CurrentUserDep, DbDep
from project_manager.core.security import create_access_token,
verify_password
from project_manager.dto.auth import Token
from project_manager.dto.user import UserResponse

router = APIRouter(prefix="/auth", tags=["Authentication"])


@router.post("/login", response_model=Token)
async def login(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
    db: DbDep
):
    # OAuth2PasswordRequestForm utilise le champ 'username' (qui contiendra
l'email de l'utilisateur)
    user = await db["users"].find_one({"email": form_data.username})

    if not user or not verify_password(form_data.password,
user["hashed_password"]):
        raise HTTPException(
```

```
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Email ou mot de passe incorrect.",
            headers={"WWW-Authenticate": "Bearer"},
        )

    if not user.get("is_active", True):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Ce compte utilisateur est désactivé."
        )

    # 'sub' (Subject) est le standard JWT pour stocker l'identifiant unique
    access_token = create_access_token(data={"sub": str(user["_id"])})
    return Token(access_token=access_token)


@router.get("/me", response_model=UserResponse)
async def read_users_me(current_user: CurrentUserDep):
    return current_user
```

5.3 Enregistrer le router dans `src/project_manager/main.py`

Ajoute simplement `auth` dans `main.py` :

```
from project_manager.routers import auth, roles, users # adapte selon les
noms exacts de tes fichiers

app.include_router(auth.router, prefix="/api/v1")
```

Test de Validation dans Swagger UI

1. Rafraîchis `[http://127.0.0.1:8000/docs](http://127.0.0.1:8000/docs)`. Un bouton vert **"Authorize"** apparaît maintenant en haut à droite.
2. Clique sur **Authorize**, renseigne dans `username` l'email d'un utilisateur créé précédemment (ex: `jean.mendy@taskflow.dev`) et son `password` (`Password123!`), puis valide.
3. Teste l'endpoint protégé **`GET /api/v1/auth/me`** : il doit te retourner ton profil complet avec son rôle imbriqué sans avoir à saisir d'ID !

Dis-moi dès que la connexion et la route `/me` fonctionnent chez toi !